



Streamlining Clinical Workflows Through  
Unified Integration

# CLINIQ API: End-to- End Clinical Operations

Designed & Presented by Luqman Cassim

Date. 2026.7.30

|          |                                             |    |
|----------|---------------------------------------------|----|
| Contents | Overview & Core Concepts                    | 01 |
|          | Setup, Configuration & Security             | 02 |
|          | Appointment Lifecycle & Clinical Logic      | 03 |
|          | Financials, Data Integrity & Error Handling | 04 |



01

# Overview & Core Concepts

# ++ CLINIQ API at a Glance

CLINIQ API delivers end-to-end clinical operations, unifying appointments, records, billing, and access for modern clinic, EMR, and patient portal solutions.

## End-to-end focus

CLINIQ API is a clinical operations backbone that manages the full appointment lifecycle, linking booking, visit, documentation, billing, and payment into one consistent flow.

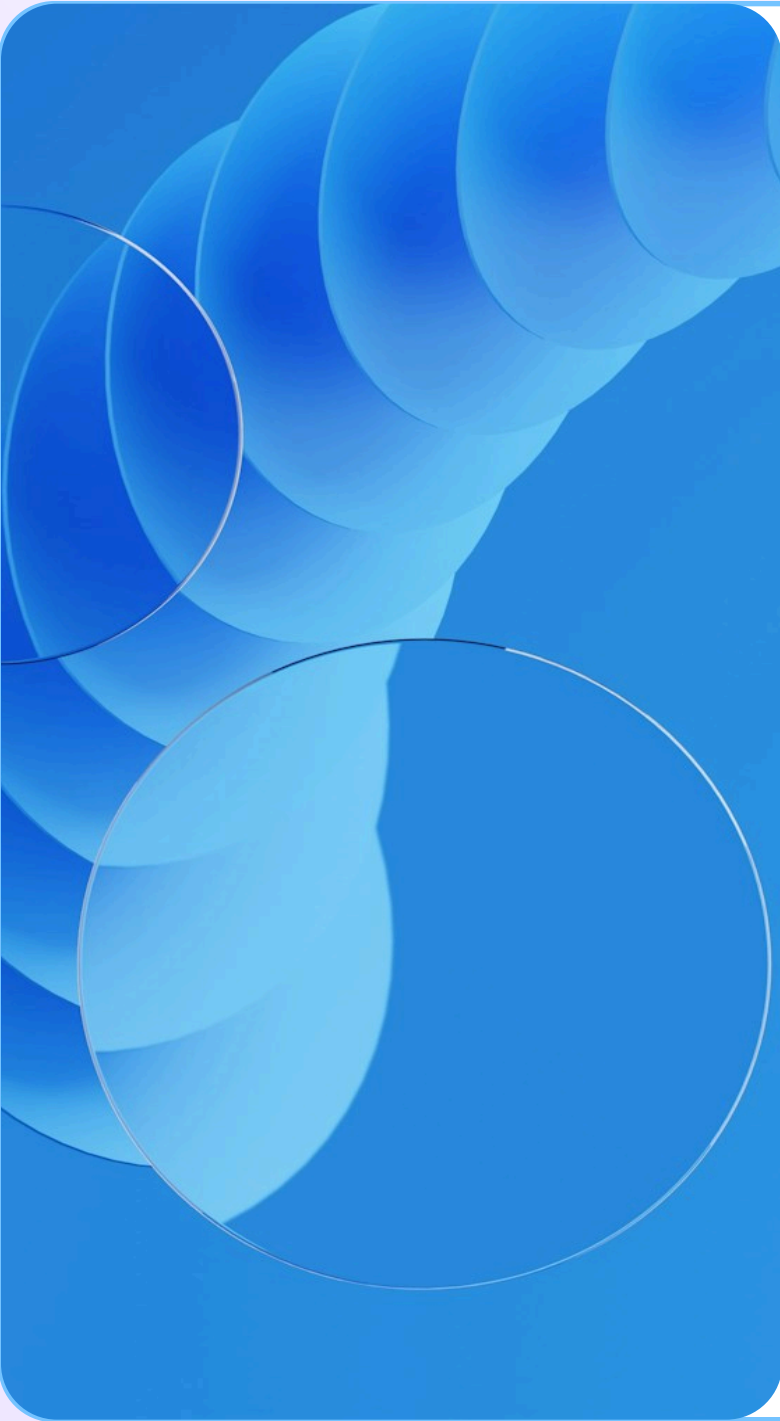
## Stack & use cases

Built on .NET 10 and SQL Server with JWT-based authentication and authorization, CLINIQ powers clinic management, EMR/practice tools, and patient portals with secure, scalable APIs.

## Core data domains

CLINIQ structures clinical data across core domains: Patients, Doctors, Appointments, Medical Records, Invoices, and Payments, enabling coherent workflows and reliable reporting.

# ++ System Architecture & Technology Stack



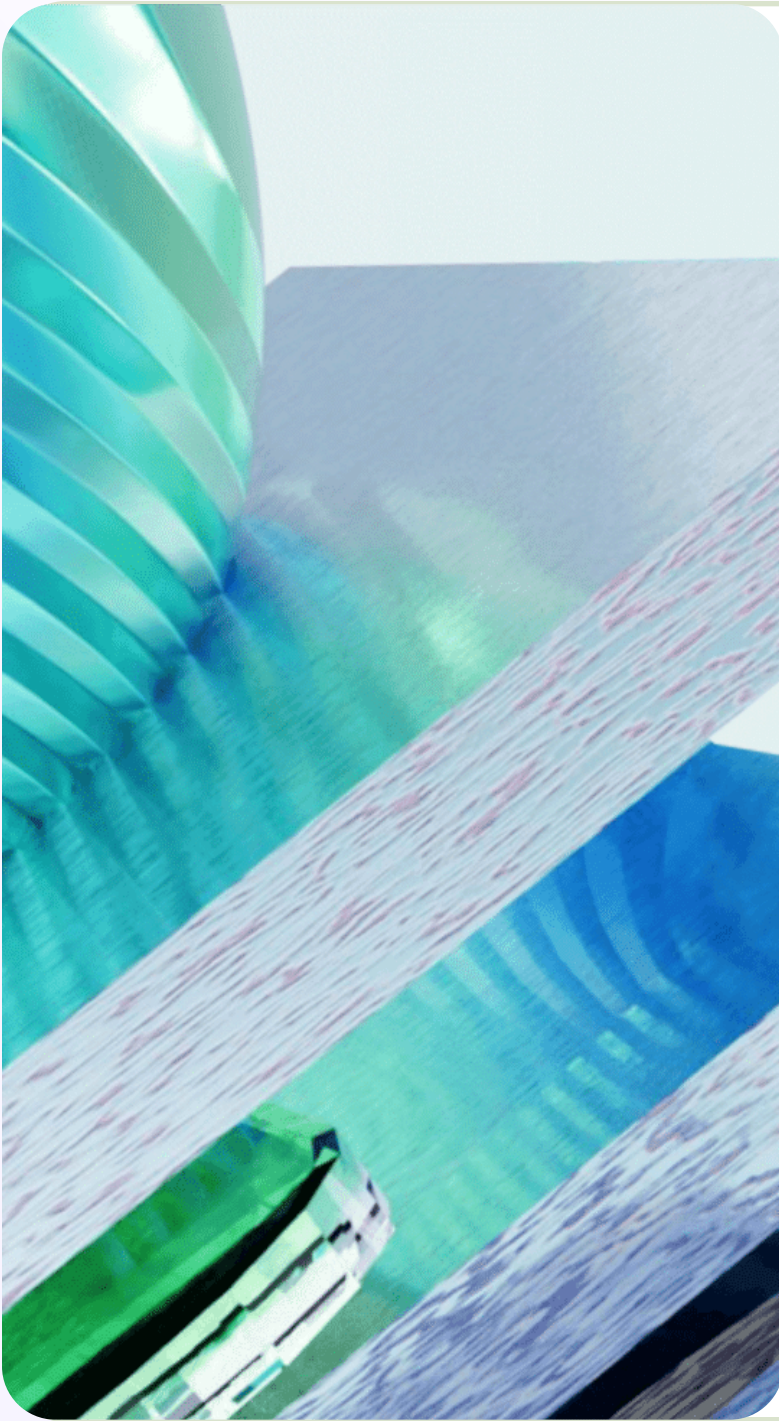
## Logical Layers & Technology Choices

Architecture separates API layer, domain services, data access and database, implemented using .NET 10 SDK, EF Core, and SQL Server as the primary transactional datastore.



## Environments, Security & Extensibility

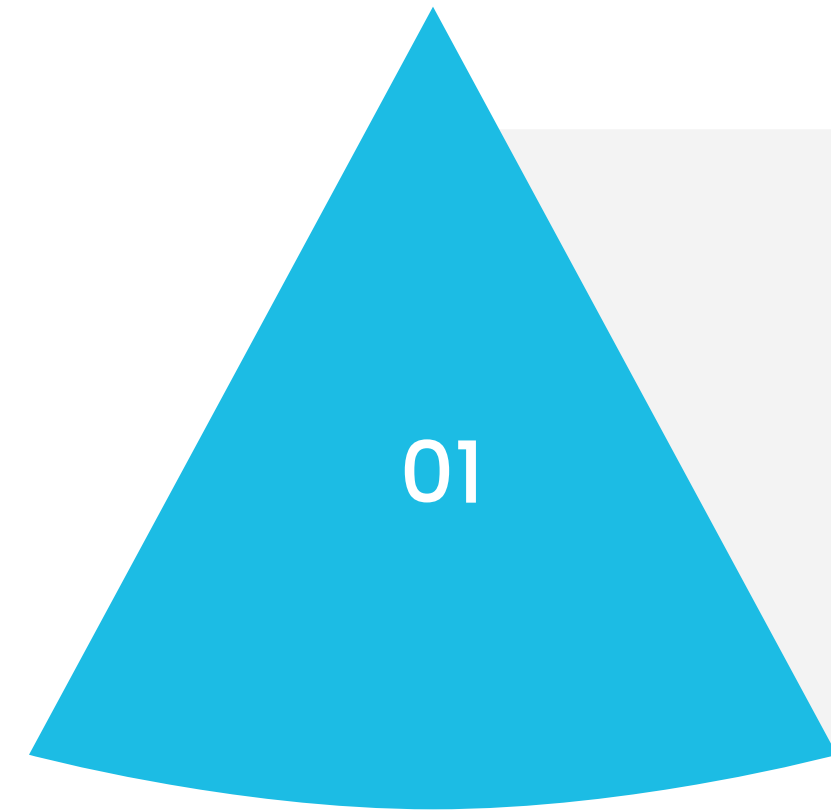
Development uses interactive /swagger interface, while production deployments emphasize JWT-based auth, role-based access control, and extensible integration with billing, notification, insurance systems.



## Containerization & Cloud Deployment

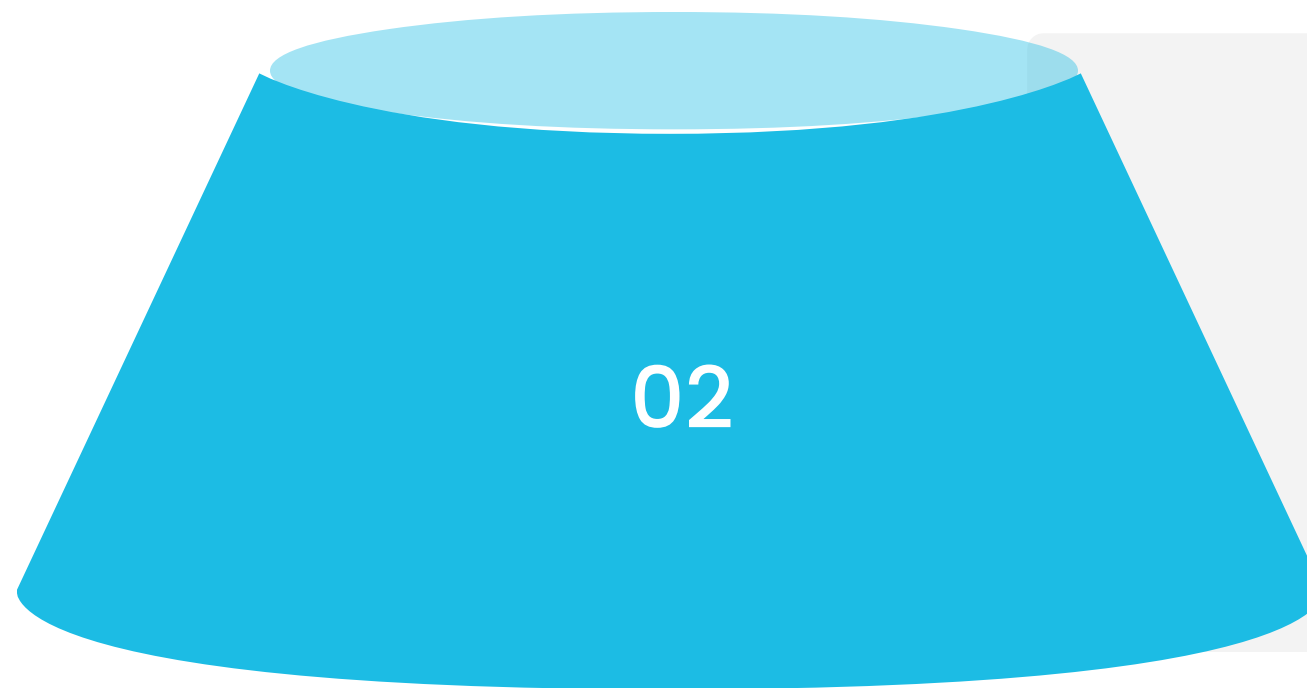
Leverages pre-configured Docker container images pre-loaded with database schemas and migration scripts, ensuring streamlined, consistent, and ready-to-publish deployments directly to Azure SQL Database.

# ++ Core Domain Model Overview



## Outcomes, Artifacts, and Traceability

Third tier holds Medical Records and Invoices, while base tier includes Payments and audit logs. All entities connect back to each appointment for end-to-end traceability.



## Pyramid of Core Clinical Entities

Appointment sits at the top, anchoring the clinical workflow. Patient and Doctor form the second tier as primary participants bound to each appointment in the lifecycle.

# ++ Clinical Workflow in One Lifecycle

01

## Canonical Flow & API Mapping

Book, update, complete, record, invoice, pay flow mapped to dedicated API endpoints, status transitions, and lifecycle states for each appointment interaction.

02

## Business Rules, Auditing & Lifecycle

Business rules enforce valid sequencing, such as completion before record; integrated audit logging, error handling, and “one appointment, one lifecycle” integrity across all operations.



## AUTH WORKFLOW

# Authentication & Authorization Roles

## JWT Bearer Token

Send a JWT in the `Authorization` request header for all protected endpoints: `Authorization: Bearer <token>`. Requests without a valid token receive **401 Unauthorized**.

## Admin

Full access across the system. Admins can link a doctor user account to a doctor profile via `PUT /api/doctors/{doctorId}/user` with a `userId` in the request body.

## Doctor & Receptionist

Doctors can record treatment decisions on in-progress appointments. Receptionists and Doctors can create medical records, invoices, and record payments alongside Admins.

## Patient

Patients have a dedicated role used for scoping access to their own data. Certain write operations are restricted to Admin, Doctor, or Receptionist roles only.

# ++ Role-Based Access Control

## Admin Permissions

Full platform oversight: system configuration, endpoint settings, user and role management, access audits, and security policies to ensure compliant clinical operations across all CLINIQ API services.

## Doctor Permissions

Clinical decision authority: manage appointment progress, approve or reject treatments, create and update medical records, and access diagnostic endpoints within defined compliance and audit constraints.

## Patient Permissions

Self-service access: book appointments, review upcoming and past visits, download invoices, complete payments, and securely view personal health information through patient-facing endpoints.

## Receptionist Permissions

Front desk workflows: create, reschedule, and cancel appointments, manage basic patient visit details, coordinate schedules, while restricted from modifying clinical records or approving treatments.

## Cross-Role Constraints

Strict separation of duties: only Doctors may approve treatments or alter clinical records; Admins configure access policies; Receptionists and Patients have limited, non-clinical write permissions.

## Typical Access Scenarios

Example flows: Patient books via scheduling API, Receptionist adjusts time, Doctor updates treatment plan, Admin audits activity; each endpoint enforces role checks and permission scopes.



03

## Appointment Life cycle & Clinical Logic

## APPOINTMENTS

# Appointment Workflow

Every appointment moves through a defined lifecycle. Status values are numeric:  
**0** Scheduled, **1** Confirmed, **2** Completed, **3** Cancelled, **4** NoShow, **5** InProgress.



### Create & Confirm

Create the appointment (Scheduled), then update its status as the patient arrives and the visit progresses through **Confirmed** and **InProgress**.



### Complete & Record

Mark the appointment **Completed** after the consultation, then create one medical record and, where applicable, one invoice for that appointment.



### Collect Payment

Record payments against the invoice until it is fully paid. The invoice status transitions to **PartiallyPaid** or **Paid** automatically as payments are received.

# Clinical Treatment Approval

The assigned doctor can record a treatment decision while an appointment is **InProgress**. The action completes the appointment, saves one medical record, and records an audit entry automatically.



## Endpoint & Role Requirement

**POST** `/api/appointments/{id}/treatment`

requires the Doctor role and a doctor user account linked to the appointment's doctor profile via PUT

`/api/doctors/{doctorId}/user`



## Medication Treatment (type 1)

Provide diagnosis, treatmentType: 1, prescription, and optional notes.

**Example:** "Cetirizine 10mg once daily for 7 days."

Records the clinical plan – does not mean medicine was dispensed.



## Surgical Treatment (type 2)

Provide diagnosis, treatmentType: 2, and surgeryDetails.

**Example:** "Refer for laparoscopic cholecystectomy assessment."

Records the referral plan; does not mean surgery was performed.



## Idempotency Constraint

A treatment decision can only be recorded once per appointment. Attempting to record it a second time returns **409 Conflict**.

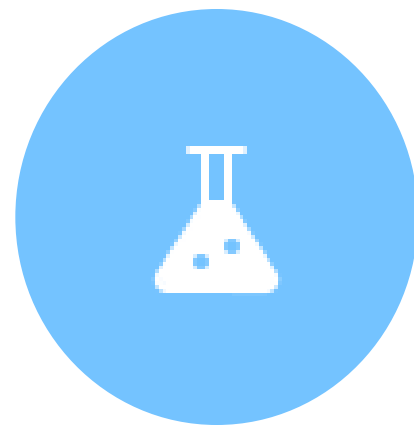
The appointment must be InProgress at the time of the request.



**Summary:** Record a treatment once, complete the appointment, save one medical record, and create an audit entry automatically.

# ++ Treatment Approval Rules & Examples

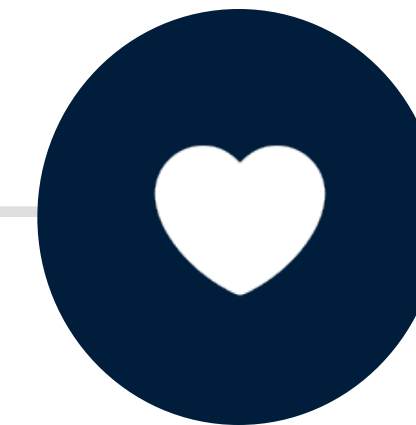
2024



## Validation & Duplication Logic

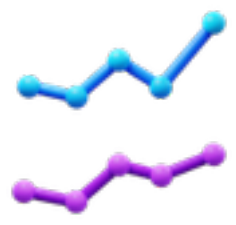
Define treatment payload validation: type, dosage, notes, required fields. Explain multiple treatments per appointment, allowed combinations, and what triggers duplicate treatment 409 Conflict.

2025



## Clinical Examples & Compliance

Illustrate with medication orders and referral documentation flows. Emphasize traceability, medico-legal documentation, audit trails, and alignment with regulatory compliance and institutional clinical governance.



04

## Financial, Data Integrity & Error Handling

# ++ Records, Invoices & Payments Relationships



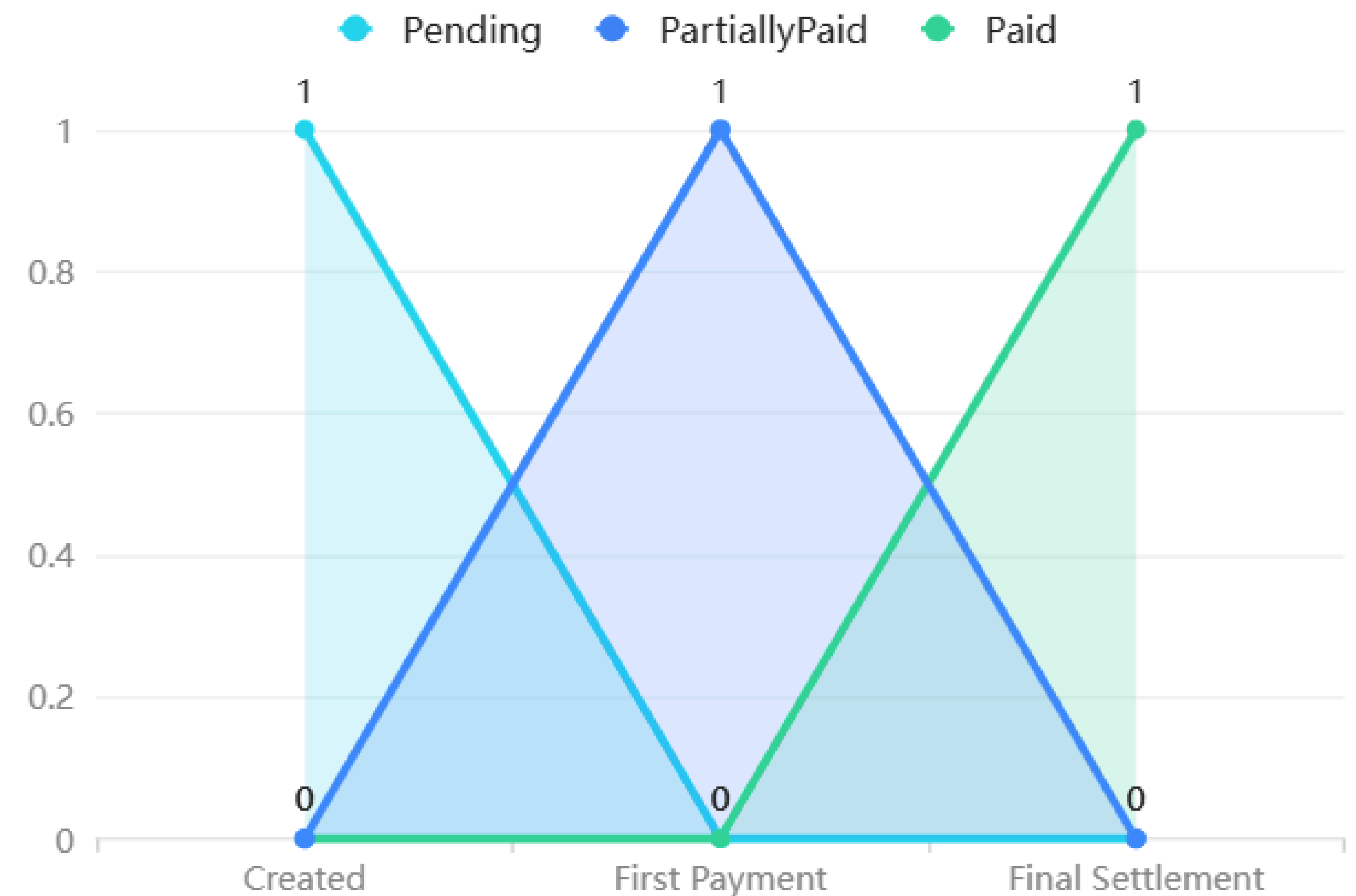
# ++ Payment Logic & Status Handling

## 01 Invoice Status & Balance Rules

Invoices progress through Pending, PartiallyPaid, and Paid statuses. Remaining balance is calculated from total minus applied payments, blocking overpayments and enforcing strict data integrity controls.

## 02 Partial Payments, Refunds & Integrations

Define how partial payments, refunds, and write-offs affect status and balance. Document status transitions and integration flows with external gateways, POS terminals, and financial reconciliation systems.



# ++ Error Handling & Conflict Scenarios

## Client

Define core client error responses: 400 for validation issues, 401 for missing or invalid authentication, and 403 for requests blocked by permission rules.

## Missing

Explain 404 Not Found when requested resources do not exist or are no longer available, guiding clients to handle missing data gracefully and inform users clearly.

## Conflict

Detail 409 Conflict scenarios such as duplicate records, invoices created twice, or records submitted before workflow completion, indicating business rule clashes.

Cover advanced integrity safeguards, including overpayment attempts and duplicate treatment submissions, treated as guarded operations that return conflict errors.

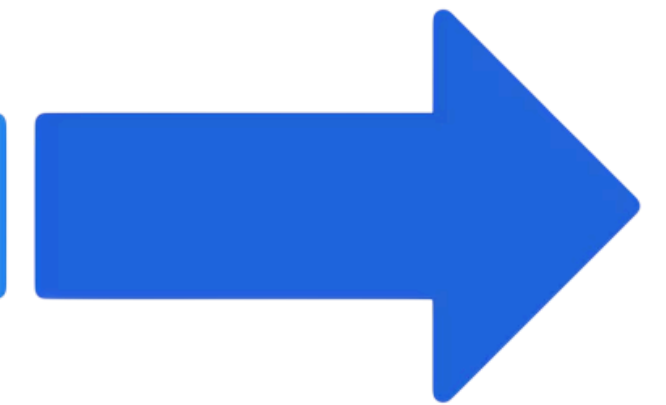
Clarify how business rule violations differ from basic client errors or permission failures, enabling precise handling paths and better operational visibility.

Recommend best practices: structured error parsing, mapping error codes to user friendly messages, logging context, and guiding users to resolve issues quickly.

## Guarded

## Types

## Practice



# ++ Putting It All Together: One Complete Lifecycle

## Lifecycle Loop & Multi-Role Participation

Book → Progress → Complete → Record → Invoice → Pay, with patient, doctor, receptionist, and admin each contributing actions and confirmations through every clinical and financial stage.

Each role's responsibilities and handoffs are clearly mapped, reducing ambiguity, improving collaboration, and ensuring accountability across the full appointment and revenue cycle.



## Data Integrity, Traceability & Scalable Operations

Strict statuses, single appointment record and invoice, and robust payment validations protect data integrity while enabling reliable reporting and financial reconciliation. Auditable, fully traceable events create predictable workflows, minimize errors, clarify ownership, and position CLINIQ as a scalable, compliant clinical operations backbone.



Date. 2026.7.30

# Thanks

A Heartfelt Appreciation for Your Support

Designed & Presented by Luqman Cassim